CSE221: Algorithms

Karatsuba's Multiplication Algorithm

Prepared by: A B M Muntasir Rahman [MUNR]



Brute Force Approach

```
    12
 x  34
------
    48
  36x
------
   408
```

Brute Force Approach

```
   12
 x  34
 ----
   48
  36x
 ----
  408
```

How many multiplications are required?

4 x 2 = 8
4 x 1 = 4
3 x 2 = 6
3 x 1 = 3

Total 4 (2^2) multiplications are required

Brute Force Approach

```
    123
  x 456
  -----
    7 3 8
  6 1 5 x
4 9 2 x x
---------
5 6 0 8 8
```

How many multiplications are required?

6 x 3 = 18
6 x 2 = 12
6 x 1 = 6
5 x 3 = 15
5 x 2 = 10
.
.
4 x 1 = 4

Total 9 (3^2) multiplications are required

Brute Force Approach

Therefore, it can be said that total $O(n^2)$ multiplications are required where n = number of digits

Divide and Conquer Approach

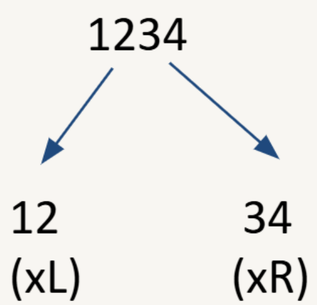
Let, **x = 1234** and **y = 5678**

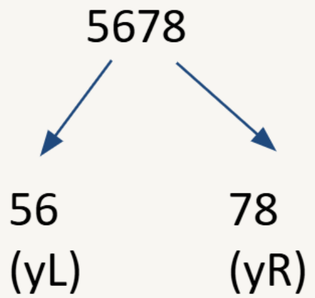
We want to find **x * y = 1234 * 5678**

x and y can represented as follows,

x = 1234 = 12 * 100 + 34 = **12 * 10^2 + 34**

y = 5678 = 56 * 100 + 78 = **56 * 10^2 + 78**





Divide and Conquer Approach

Therefore,

$x = 1234 = 12 * 10^2 + 34 = xL * 10^{n/2} + xR$
$y = 5678 = 56 * 10^2 + 78 = yL * 10^{n/2} + yR$

Now, $x * y = (12 * 10^2 + 34) * (56 * 10^2 + 78)$

$= 12*56*10^4 + 12*78*10^2+34*56*10^2+34*78$

$= 12*56*10^4 + (12*78 + 34*56)*10^2 + 34*78$

We can generalize the formula in terms of xL, xR, yL and yR
$x * y = xL*yL*10^n + (xL*yR + xR*yL)*10^{n/2} + xR*yR$

Divide and Conquer Approach

$$x * y = xL*yL*10^{n} + (xL*yR + xR*yL)*10^{n/2} + xR*yR$$

Entire problem has been divided into 4 subproblems

However, this is not efficient yet 😢

Workaround?

Divide and Conquer Approach

Recall $x * y = 12*56*10^4 + (12*78 + 34*56)*10^2 + 34*78$

Now, **(12*78 + 34*56) = (12 + 34) (56 + 78) - (12*56) - (34*78)**
= c - a - b

$x * y = 12*56*10^4 +$ **((12 + 34) (56 + 78) - (12*56) - (34*78))**$*10^2 + 34*78$

Generalizing, we can say that,

$x*y = a*10^n + (c - a - b)*10^{n/2}+b$

where a = xL*yL, b = xR*yR and c = ((xL+xR)(yL+yR) - (xL*yL) - (xR*yR))

This is called Karatsuba's Algorithm for Multiplication!

Time Complexity Analysis

```
karatsuba(x, y)
  if len(str(x)) == 1 or len(str(y)) == 1
     return x*y
  n = max(len(str(x)), len(str(y)))
  m = (n + 1) // 2
  p = 10**m
  xL = x // p
  xR = x % p
  yL = y // p
  yR = y % p
  a = karatsuba(xL, yL)                    ──────────►  T(n/2)
  b = karatsuba(xR, yR)                    ──────────►  T(n/2)
  c = karatsuba(xL + xR, yL + yR)          ──────────►  T(n/2) + O(n)
  return a * (10**2m) + (c - a - b) * (10**m) + b
```

Time Complexity Analysis

Therefore,

$T(n) = 3T(n/2) + O(n)$

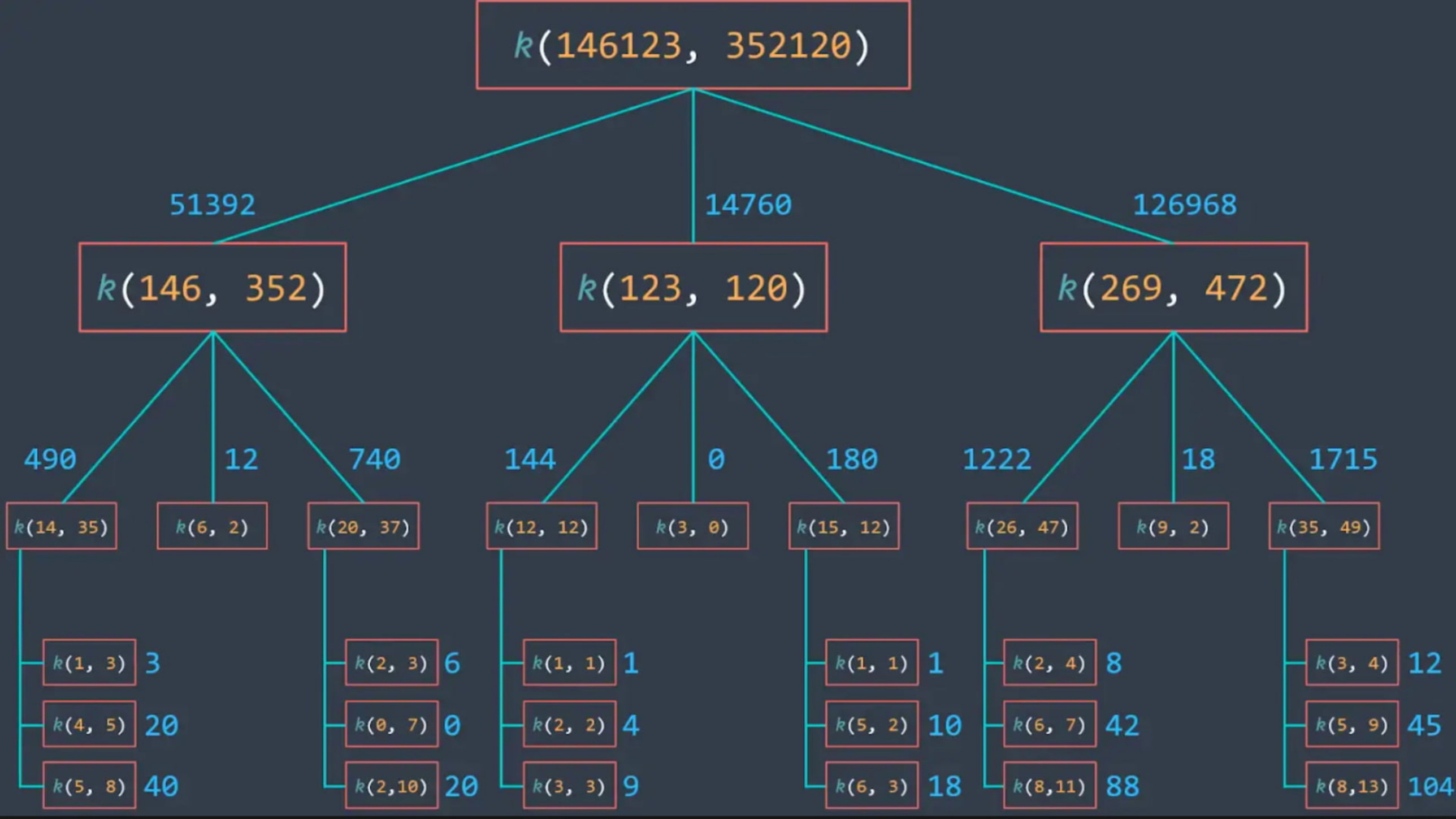
Applying Master's theorem,

$a = 3$ and $b^k = 2$, therefore $a > b^k$

$T(n) = O(n^{(\log_b a)}) = O(n^{(\log_2 3)}) =$ **$O(n^{1.59})$** which is better than $O(n^2)$

Use cases

Domain	Where Used	When Used	Next Algorithms After Karatsuba
Python	`int` type	For large ints (>70 digits)	Toom–Cook, FFT
Java	`BigInteger`	For ~100–1000 bit numbers	Toom–Cook
GMP Library	Arbitrary-precision arithmetic	Medium-size integers	Toom–Cook, FFT
Cryptography	RSA, ECC, DH	Modular exponentiation	Optimized big-int ops


k(146123, 352120)
51392
14760
126968
k(146, 352)
k(123, 120)
k(269, 472)
490
12
740
144
0
180
1222
18
1715
k(14, 35)
k(6, 2)
k(20, 37)
k(12, 12)
k(3, 0)
k(15, 12)
k(26, 47)
k(9, 2)
k(35, 49)
k(1, 3) 3
k(4, 5) 20
k(5, 8) 40
k(2, 3) 6
k(0, 7) 0
k(2,10) 20
k(1, 1) 1
k(2, 2) 4
k(3, 3) 9
k(1, 1) 1
k(5, 2) 10
k(6, 3) 18
k(2, 4) 8
k(6, 7) 42
k(8,11) 88
k(3, 4) 12
k(5, 9) 45
k(8,13) 104